

MarketHub

Fiche explicative — Ce qu'on a construit aujourd'hui

Alphonse (Backend) · Elisabeth (Frontend) · Dovane (Full-stack)

Dimanche 4 mai 2026 — Jour 2

1. C'est quoi MarketHub ? (version simple)

Imagine un site comme Leboncoin, mais pour les étudiants.

Un étudiant qui sait faire des sites web peut proposer ses services.

Un autre étudiant qui a besoin d'un site peut acheter ce service.

Ils peuvent discuter en temps réel, payer, laisser un avis...

Tout ça dans une seule application qu'on construit ensemble !

2. Ce qu'on a fait aujourd'hui

Etape 1 — Préparer les outils

Avant de construire une maison, il faut les bons outils. Pour une application, c'est pareil.

PHP 8.2	Le langage de programmation. C'est comme la langue qu'on parle pour parler à l'ordinateur.
Composer	Le livreur de paquets. Il installe automatiquement tout ce dont on a besoin.
Symfony 5.4	Le cadre de travail. Comme un moule de gâteau qui donne la forme à notre application.
Git Bash	Le terminal qu'on utilise pour taper les commandes. Plus fiable que PowerShell sur Windows.

Etape 2 — Installer Symfony

Symfony c'est notre 'boîte à outils' pour faire une API. On a installé la version 5.4 — la plus stable avec notre PHP.

Commande magique qu'on a tapee :

```
composer create-project symfony/skeleton:"5.4.*" backend --no-interaction
```

Ce que ca a cree : un dossier backend/ avec toute la structure de notre application.

Dossiers crees : bin/ config/ migrations/ public/ src/ var/ vendor/

Etape 3 — Creer la base de donnees

La base de donnees c'est le ' tiroir ' ou on range toutes les informations de l'application.

On a configure le fichier .env pour dire a Symfony ou trouver MySQL :

```
DATABASE_URL="mysql://root:@127.0.0.1:3306/markethub"
```

Ca veut dire : connecte-toi a MySQL sur mon ordinateur, utilise la base de donnees 'markethub'.

Ensuite on a cree les 7 tables en base de donnees :

user	Stocke tous les utilisateurs (nom, email, mot de passe, role...)
service	Stocke tous les services proposes (titre, description, prix...)
order	Stocke toutes les commandes passees par les acheteurs
review	Stocke les avis et notes laisses apres une commande
message	Stocke tous les messages envoyes entre utilisateurs
category	Stocke les categories de services (dev, design, redaction...)
notification	Stocke les notifications envoyees aux utilisateurs

Etape 4 — Creer les entites Doctrine

Une entite c'est comme un 'formulaire' qui dit a Symfony comment ranger les donnees en base.

Exemple simple : l'entite User dit :

- id = un numero unique pour chaque utilisateur
- email = l'adresse email (obligatoire, unique)
- password = le mot de passe (stocke de facon securisee, jamais en clair)
- role = buyer (acheteur) ou seller (vendeur) ou admin
- username = le pseudo de l'utilisateur

On a cree 7 entites : User, Service, Order, Review, Message, Category, Notification

Etape 5 — Créer les endpoints API

Un endpoint c'est une 'porte d'entree' de notre application. Chaque porte fait une chose precise.

Analogie simple :

C'est comme les guichets d'une banque.

Guichet 1 : 'Je veux ouvrir un compte' -> POST /api/auth/register

Guichet 2 : 'Je veux me connecter' -> POST /api/auth/login

Guichet 3 : 'Je veux voir les services' -> GET /api/services

Guichet 4 : 'Je veux passer commande' -> POST /api/orders

Les endpoints qu'on a crees :

POST /api/auth/register	Creer un nouveau compte utilisateur
POST /api/auth/login	Se connecter et recevoir un token JWT
GET /api/services	Voir la liste des services (avec filtres)
GET /api/services/{id}	Voir le detail d'un service
POST /api/services	Creer un nouveau service (vendeur)
POST /api/orders	Passer une commande
GET /api/orders/my	Voir ses propres commandes
POST /api/reviews	Laisser un avis

Etape 6 — Le JWT (la cle secrete)

JWT = JSON Web Token. C'est comme un badge d'accès.

Comment ca marche :

1. Tu te connectes avec ton email + mot de passe
2. Le serveur verifie que c'est bien toi
3. Il te donne un 'badge' (le token JWT)
4. Pour chaque action, tu montres ton badge
5. Le serveur sait que c'est toi sans que tu retapes ton mot de passe

Le badge expire apres 1 heure (access token = 15 minutes, refresh token = 7 jours).

C'est comme une carte de cinema valable pour la journee.

3. Les tests qu'on a valides

Test 1 — Inscription (SUCCEs) :

Commande : `curl -X POST http://localhost:8000/api/auth/register ...`

Reponse : `{"message":"Compte cree avec succes","user":{"id":1,"email":"test@test.com"}}`

Test 2 — Connexion JWT (SUCCEs) :

Commande : `curl -X POST http://localhost:8000/api/auth/login ...`

Reponse : `{"token":"eyJ0eXAiOiJKV1Qi..."}`

Ces 2 tests prouvent que :

- Notre API fonctionne
- La base de donnees est bien connectee
- Les mots de passe sont bien securises (hasches)
- Le JWT est bien genere

4. Vocabulaire cle si le prof pose des questions

API (Application Programming Interface)

C'est le 'serveur' de notre application. Le frontend (Vue.js) lui pose des questions, il repond avec des donnees. Comme un serveur dans un restaurant : tu commandes, il apporte.

REST (Representational State Transfer)

C'est la facon dont on structure notre API. Chaque action a une URL et une methode (GET = lire, POST = creer, PUT = modifier, DELETE = supprimer).

Symfony

C'est le framework PHP qu'on utilise pour construire notre API. Un framework c'est un ensemble de regles et d'outils qui nous aident a coder plus vite et de facon organisee.

Doctrine ORM

C'est l'outil qui fait le lien entre notre code PHP et la base de donnees MySQL. On ecrit du PHP, Doctrine traduit en SQL. Comme un traducteur.

JWT (JSON Web Token)

C'est le systeme d'authentification qu'on utilise. Quand tu te connectes, tu recois un token (une chaine de caracteres chiffree). Tu l'envoies avec chaque requete pour prouver qui tu es.

WebSocket

C'est la technologie pour la messagerie en temps reel. Contrairement a une API classique ou le client demande et le serveur repond, le WebSocket garde une connexion ouverte en permanence. Comme un appel telephonique qui ne raccroche pas.

Docker Compose

C'est l'outil qu'on utilisera pour lancer toute notre stack (frontend + backend + BDD + WebSocket) avec une seule commande. Comme un interrupteur unique qui allume toute la maison.

5. L'architecture en 3 couches

Notre application est decoupee en 3 couches independantes :

COUCHE 1 - PRESENTATION (Frontend Vue.js) - Elisabeth

C'est ce que l'utilisateur voit : les pages, les boutons, les formulaires.

Elle ne sait pas comment la base de donnees est organisee.

COUCHE 2 - METIER (Backend Symfony) - Alphonse + Dovane

C'est le cerveau de l'application. Elle applique les regles metier :

'Un vendeur ne peut pas commander son propre service'

'On ne peut laisser un avis que si la commande est terminee'

COUCHE 3 - DONNEES (MySQL + Doctrine) - Alphonse

C'est la memoire de l'application. Elle stocke tout.

Regle d'or : chaque couche ne parle qu'a celle directement en dessous.

6. Recap Jour 2 — Ce qui est fait

- ✓ Symfony 5.4 installe et configure
- ✓ Base de donnees markethub creee dans MySQL
- ✓ 7 entites Doctrine creees (User, Service, Order, Review, Message, Category, Notification)
- ✓ 7 tables generees automatiquement en base de donnees
- ✓ JWT configure (cles privee et publique generees)
- ✓ Endpoint POST /api/auth/register FONCTIONNE
- ✓ Endpoint POST /api/auth/login FONCTIONNE et retourne un JWT
- ✓ Controllers AuthController, ServiceController, OrderController, ReviewController crees

✓ Security.yaml configure pour proteger les routes

Ce qui reste pour demain (Jour 3) :

- Tester tous les endpoints avec Postman
- Frontend Vue.js connecte au backend (Elisabeth)
- WebSocket Ratchet pour la messagerie (Dovane)
- Docker Compose pour lancer toute la stack
- Tests e2e Cypress du parcours acheteur

Bonne continuation a toute l'equipe ! ☐

MarketHub — Jour 2 — 4 mai 2026